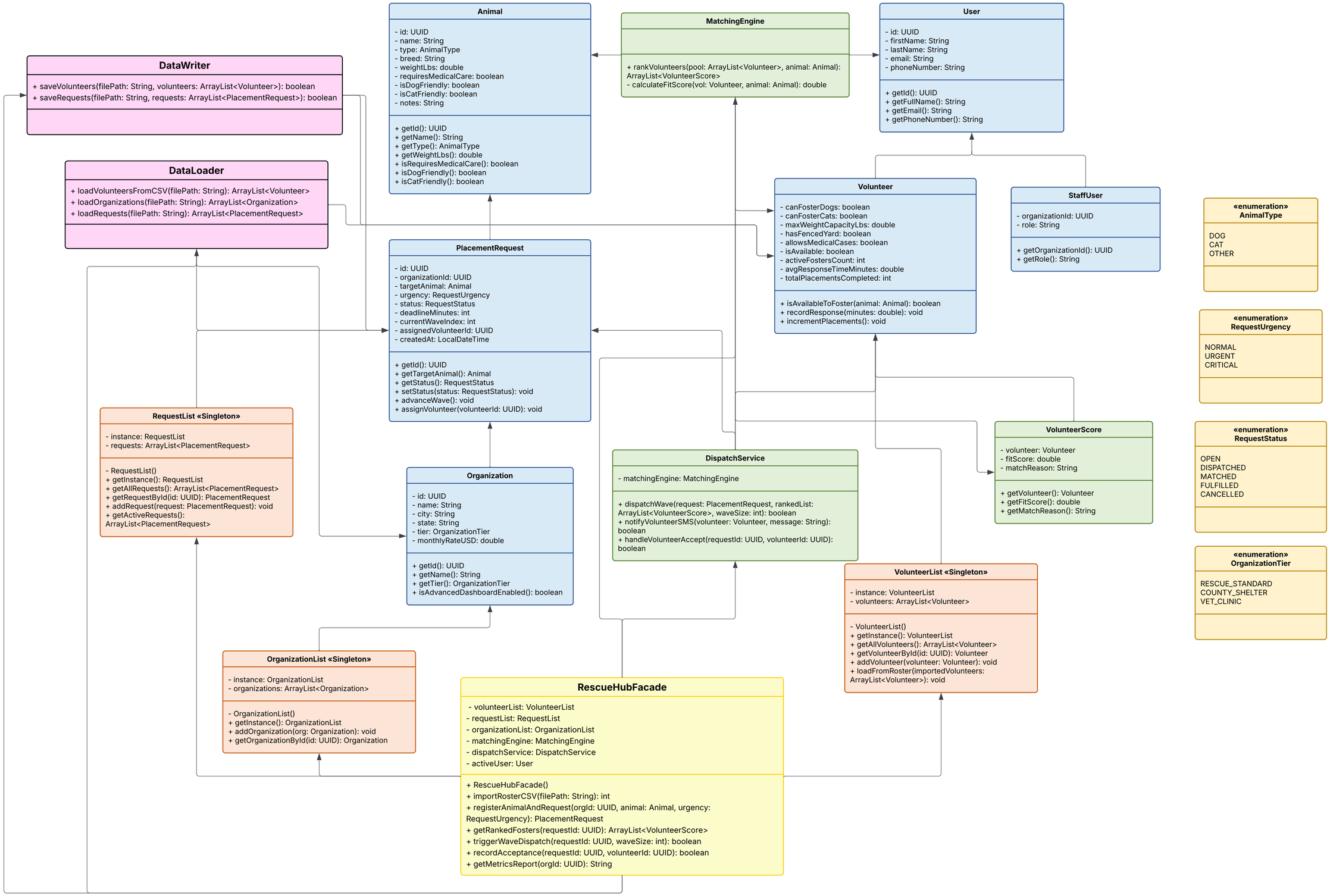

Animal
- id: UUID
- name: String
- type: AnimalType
- breed: String
- weightLbs: double
- requiresMedicalCare: boolean
- isDogFriendly: boolean
- isCatFriendly: boolean
- notes: String
+ getId(): UUID
+ getName(): String
+ getType(): AnimalType
+ getWeightLbs(): double
+ isRequiresMedicalCare(): boolean
+ isDogFriendly(): boolean
+ isCatFriendly(): boolean
MatchingEngine
+ rankVolunteers(pool: ArrayList<Volunteer>, animal: Animal): ArrayList<VolunteerScore>
- calculateFitScore(vol: Volunteer, animal: Animal): double
User
- id: UUID
- firstName: String
- lastName: String
- email: String
- phoneNumber: String
+ getId(): UUID
+ getFullName(): String
+ getEmail(): String
+ getPhoneNumber(): String
DataWriter
+ saveVolunteers(filePath: String, volunteers: ArrayList<Volunteer>): boolean
+ saveRequests(filePath: String, requests: ArrayList<PlacementRequest>): boolean
DataLoader
+ loadVolunteersFromCSV(filePath: String): ArrayList<Volunteer>
+ loadOrganizations(filePath: String): ArrayList<Organization>
+ loadRequests(filePath: String): ArrayList<PlacementRequest>
Volunteer
- canFosterDogs: boolean
- canFosterCats: boolean
- maxWeightCapacityLbs: double
- hasFencedYard: boolean
- allowsMedicalCases: boolean
- isAvailable: boolean
- activeFostersCount: int
- avgResponseTimeMinutes: double
- totalPlacementsCompleted: int
+ isAvailableToFoster(animal: Animal): boolean
+ recordResponse(minutes: double): void
+ incrementPlacements(): void
StaffUser
- organizationId: UUID
- role: String
+ getOrganizationId(): UUID
+ getRole(): String
«enumeration»
AnimalType
DOG
CAT
OTHER
«enumeration»
RequestUrgency
NORMAL
URGENT
CRITICAL
PlacementRequest
- id: UUID
- organizationId: UUID
- targetAnimal: Animal
- urgency: RequestUrgency
- status: RequestStatus
- deadlineMinutes: int
- currentWaveIndex: int
- assignedVolunteerId: UUID
- createdAt: LocalDateTime
+ getId(): UUID
+ getTargetAnimal(): Animal
+ getStatus(): RequestStatus
+ setStatus(status: RequestStatus): void
+ advanceWave(): void
+ assignVolunteer(volunteerId: UUID): void
RequestList «Singleton»
- instance: RequestList
- requests: ArrayList<PlacementRequest>
- RequestList()
+ getInstance(): RequestList
+ getAllRequests(): ArrayList<PlacementRequest>
+ getRequestById(id: UUID): PlacementRequest
+ addRequest(request: PlacementRequest): void
+ getActiveRequests(): ArrayList<PlacementRequest>
Organization
- id: UUID
- name: String
- city: String
- state: String
- tier: OrganizationTier
- monthlyRateUSD: double
+ getId(): UUID
+ getName(): String
+ getTier(): OrganizationTier
+ isAdvancedDashboardEnabled(): boolean
DispatchService
- matchingEngine: MatchingEngine
+ dispatchWave(request: PlacementRequest, rankedList: ArrayList<VolunteerScore>, waveSize: int): boolean
+ notifyVolunteerSMS(volunteer: Volunteer, message: String): boolean
+ handleVolunteerAccept(requestId: UUID, volunteerId: UUID): boolean
VolunteerScore
- volunteer: Volunteer
- fitScore: double
- matchReason: String
+ getVolunteer(): Volunteer
+ getFitScore(): double
+ getMatchReason(): String
«enumeration»
RequestStatus
OPEN
DISPATCHED
MATCHED
FULFILLED
CANCELLED
«enumeration»
OrganizationTier
RESCUE_STANDARD
COUNTY_SHELTER
VET_CLINIC
VolunteerList «Singleton»
- instance: VolunteerList
- volunteers: ArrayList<Volunteer>
- VolunteerList()
+ getInstance(): VolunteerList
+ getAllVolunteers(): ArrayList<Volunteer>
+ getVolunteerById(id: UUID): Volunteer
+ addVolunteer(volunteer: Volunteer): void
+ loadFromRoster(importedVolunteers: ArrayList<Volunteer>): void
OrganizationList «Singleton»
- instance: OrganizationList
- organizations: ArrayList<Organization>
- OrganizationList()
+ getInstance(): OrganizationList
+ addOrganization(org: Organization): void
+ getOrganizationById(id: UUID): Organization
RescueHubFacade
- volunteerList: VolunteerList
- requestList: RequestList
- organizationList: OrganizationList
- matchingEngine: MatchingEngine
- dispatchService: DispatchService
- activeUser: User
+ RescueHubFacade()
+ importRosterCSV(filePath: String): int
+ registerAnimalAndRequest(orgId: UUID, animal: Animal, urgency: RequestUrgency): PlacementRequest
+ getRankedFosters(requestId: UUID): ArrayList<VolunteerScore>
+ triggerWaveDispatch(requestId: UUID, waveSize: int): boolean
+ recordAcceptance(requestId: UUID, volunteerId: UUID): boolean
+ getMetricsReport(orgId: UUID): String